



Network Policies

Securing Pod-to-Pod Communication — Module 8 of
13

Enterprise EKS Platform





Module Agenda

- Kubernetes Networking Fundamentals
- Network Policy Basics & Anatomy
- Ingress and Egress Rules
- Common Policy Patterns
- CNI and Network Policy Providers
- Real-World Security Strategies
- Debugging & Monitoring Network Policies



Kubernetes Networking Fundamentals

Understanding the Flat Network Model



The Kubernetes Flat Network

Core Networking Requirement

Every pod gets its own IP address, and **every pod can reach every other pod** without NAT — across nodes, across namespaces.

- **Pod-to-Pod:** Direct communication via cluster-internal IPs
- **Cross-Namespace:** No isolation boundary by default
- **Cross-Node:** CNI plugin handles overlay or native routing
- **DNS Resolution:** Every service discoverable via CoreDNS

The Security Problem: Default Allow

Default Behavior: Every pod can communicate with every other pod on every port. There are no firewalls between workloads.

Without Network Policies

- Compromised frontend reaches database directly
- Dev namespace can access production
- Lateral movement and data exfiltration are trivial

With Network Policies

- Frontend only reaches its API server
- Namespaces isolated; egress restricted
- Blast radius is contained



Network Policy Basics

Anatomy, Selectors, and Rules



NetworkPolicy Anatomy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api-server-policy, namespace: production }
spec:
  podSelector:           # Which pods this policy applies to
    matchLabels: { app: api-server }
  policyTypes: [Ingress, Egress]
  ingress:               # Allowed inbound traffic
    - from:
      - podSelector: { matchLabels: { app: frontend } }
        ports: [{ protocol: TCP, port: 8080 }]
  egress:                # Allowed outbound traffic
    - to:
      - podSelector: { matchLabels: { app: database } }
        ports: [{ protocol: TCP, port: 5432 }]
```

Pod Selectors and Label Matching

matchLabels (AND logic)

```
podSelector:  
  matchLabels:  
    app: api-server  
    tier: backend  
# Must match ALL labels
```

matchExpressions (flexible)

```
podSelector:  
  matchExpressions:  
    - key: tier  
      operator: In  
      values: [backend, middleware]  
    - key: env  
      operator: NotIn  
      values: [test]
```

Empty podSelector {} — Selects *all* pods in the namespace. This is how you create default deny policies.



Ingress Rules: Controlling Inbound Traffic

```
ingress:
  # Rule 1: Allow from pods with label app=frontend
  - from:
    - podSelector: { matchLabels: { app: frontend } }
  # Rule 2: Allow from monitoring namespace
  - from:
    - namespaceSelector: { matchLabels: { purpose: monitoring } }
  # Rule 3: Allow from corporate VPN CIDR
  - from:
    - ipBlock:
      cidr: 10.0.0.0/8
      except: [10.0.5.0/24]
```

OR vs AND: Separate list items under `from:` are OR'd. Multiple selectors *within* a single list item are AND'd.



Egress Rules: Controlling Outbound Traffic

```
egress:
  # Allow DNS resolution (critical!)
  - to: [{ namespaceSelector: {} }]
    ports: [{ protocol: UDP, port: 53 }, { protocol: TCP, port: 53 }]
  # Allow traffic to database pods
  - to:
    - podSelector: { matchLabels: { app: postgres } }
    ports: [{ protocol: TCP, port: 5432 }]
  # Allow external API calls
  - to:
    - ipBlock: { cidr: 203.0.113.0/24 }
    ports: [{ protocol: TCP, port: 443 }]
```

Never forget DNS! If you restrict egress, you must explicitly allow UDP/TCP port 53 or your pods cannot resolve service names.



Port-Level Controls

Specific Ports

```
ports:
- protocol: TCP
  port: 8080
- protocol: TCP
  port: 8443
- protocol: TCP
  port: 9090
```

Port Ranges (K8s 1.25+)

```
ports:
- protocol: TCP
  port: 8000
  endPort: 9000
```

Covers all ports from 8000 to 9000 inclusive.

Named Ports

```
ports:
- protocol: TCP
  port: http # Matches the named port in the target pod spec
```

Named ports reference the containerPort name, decoupling policy from hard-coded numbers.



Common Policy Patterns

Recommended Patterns

Default Deny Policies

Deny All Ingress

```
# kind: NetworkPolicy
# metadata: { name: default-deny-ingress, namespace: production }
spec:
  podSelector: {}      # Selects ALL pods in namespace
  policyTypes:
    - Ingress
  # No ingress rules = deny all inbound traffic
```

Deny All Egress

```
# kind: NetworkPolicy
# metadata: { name: default-deny-egress, namespace: production }
spec:
  podSelector: {}
  policyTypes:
    - Egress
  # No egress rules = deny all outbound traffic
```

Best Practice: Apply both default-deny policies to every namespace first, then layer on allow rules. Denying egress breaks DNS -- always pair with a DNS allow policy.

Companion: Allow DNS with Egress Deny

```
# Deploy alongside default-deny-egress
# kind: NetworkPolicy
# metadata: { name: allow-dns, namespace: production }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to: [{ namespaceSelector: {} }]
      ports:
        - { protocol: UDP, port: 53 }
        - { protocol: TCP, port: 53 }
```

Tip: Some teams scope DNS to kube-system only, but allowing any namespace is safer for custom DNS setups.

Allow Specific Namespace Communication

```
# kind: NetworkPolicy
# metadata: { name: allow-from-monitoring, namespace: production }
spec:
  podSelector: {}
  policyTypes: [Ingress]
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: monitoring
  ports:
    - { protocol: TCP, port: 9090 } # Prometheus metrics
    - { protocol: TCP, port: 8080 } # Health checks
```

Allow Specific Pod-to-Pod Communication

Three-Tier Application Pattern

Frontend → API Server → Database — each tier only talks to its neighbors.

```
# Allow API server to receive from frontend only
# kind: NetworkPolicy
# metadata: { name: api-allow-frontend, namespace: production }
spec:
  podSelector: { matchLabels: { tier: api } }
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector: { matchLabels: { tier: frontend } }
      ports: [{ protocol: TCP, port: 8080 }]
```



Database Isolation Pattern

```
# kind: NetworkPolicy
# metadata: { name: database-isolation, namespace: production }
spec:
  podSelector: { matchLabels: { app: postgres, tier: database } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - podSelector: { matchLabels: { tier: api } }
      ports: [{ protocol: TCP, port: 5432 }]
  egress:
    # DNS only – database should not initiate connections
    - to: [{ namespaceSelector: {} }]
      ports: [{ protocol: UDP, port: 53 }, { protocol: TCP, port: 53 }]
```



CNI & Network Policy Providers

Implementation Behind the API

CNI Provider Comparison

Feature	Calico	Cilium	AWS VPC CNI
K8s NetworkPolicy	✓	✓	✓ (v1.25+)
Global Policies	✓	✓	✗
L7 Filtering (HTTP)	✓ (Enterprise)	✓	✗
DNS-Based Rules	✓	✓	✗
Host-Level Policies	✓	✓	✗
Encryption (WireGuard)	✓	✓	✗
Flow Logging	✓	✓ (Hubble)	VPC Flow Logs



Calico Network Policies

```
apiVersion: projectcalico.org/v3
kind: GlobalNetworkPolicy
metadata: { name: deny-external-egress }
spec:
  selector: env == 'production'
  types: [Egress]
  egress:
    - action: Allow          # Cluster-internal traffic
      destination: { nets: [10.0.0.0/8] }
    - action: Allow          # DNS
      protocol: UDP
      destination: { ports: [53] }
    - action: Deny          # Everything else
```

AWS VPC CNI & Network Policy on EKS

Since EKS 1.25, the VPC CNI plugin natively enforces NetworkPolicy using eBPF.

- **Enable:** Update vpc-cni add-on with `enableNetworkPolicy: true`
- **eBPF-based:** Kernel-level enforcement with minimal latency
- **Standard API:** Uses native `networking.k8s.io/v1` resources
- **VPC Flow Logs:** AWS-native network flow visibility
- **Limitation:** L3/L4 only — no extended CRDs like Calico/Cilium

Need L7 or global policies? Run Calico as a policy-only overlay alongside VPC CNI.



Cilium: L7-Aware Network Policies

```
# kind: CiliumNetworkPolicy
# metadata: { name: api-l7-policy, namespace: production }
spec:
  endpointSelector: { matchLabels: { app: api-server } }
  ingress:
    - fromEndpoints:
        - matchLabels: { app: frontend }
      toPorts:
        - ports: [{ port: "8080", protocol: TCP }]
          rules:
            http:
              - { method: GET, path: "/api/v1/.*" }
              - { method: POST, path: "/api/v1/orders" }
```

L7 Filtering: Cilium restricts by HTTP method, path, headers, and gRPC method — far beyond standard NetworkPolicy.



Real-World Security Strategies

From Policy to Practice



Zero-Trust Networking in Kubernetes

Zero-Trust Principle

Never trust, always verify. Every connection is denied unless explicitly permitted, regardless of source location.

1. Default Deny

- Apply to every namespace
- Both ingress and egress
- Automate via admission controller

2. Least Privilege

- Allow only required paths
- Restrict ports precisely
- Scope to exact labels

3. Verify & Audit

- Log denied connections
- Review policies in CI/CD
- Test with network probes

Microsegmentation Strategies

Namespace-Level Segmentation

- Isolate by environment (dev/staging/prod)
- Isolate by team or business unit
- Broad but effective first step

```
# Block all cross-namespace traffic
ingress:
  - from:
    - podSelector: {} # Same ns only
```

Application-Level Segmentation

- Isolate by microservice tier
- Per-service egress restrictions
- Granular but higher maintenance

```
# Only specific pod-to-pod
ingress:
  - from:
    - podSelector:
        matchLabels:
          app: checkout
```

Debugging Network Policies

- **Forgotten DNS:** Egress policies that don't allow port 53 break all name resolution silently
- **OR vs AND confusion:** Two selectors in one from: item are AND'd; separate from: items are OR'd
- **CNI doesn't enforce:** Some CNI plugins (e.g., Flannel) ignore NetworkPolicy entirely
- **Namespace-scoped:** A policy in namespace A cannot affect pods in namespace B

Debugging — kubectl Commands

Policy Inspection

```
# List policies affecting a pod
kubectl get networkpolicy -n prod

# Describe policy details
kubectl describe netpol \
  api-server-policy -n prod

# Check pod labels match
kubectl get pods -n prod \
  --show-labels | grep api

# Validate YAML before apply
kubectl apply --dry-run=server \
  -f policy.yaml
```

Debugging — Network Testing

Connectivity Testing

```
# Test connectivity from a pod
kubectl run test-conn --rm -it \
  --image=busybox -- wget -qO- \
  --timeout=3 http://api-server:8080

# Test DNS resolution
kubectl run test-dns --rm -it \
  --image=busybox -- nslookup \
  api-server.production.svc

# Netcat port test
kubectl exec -it debug-pod -- \
  nc -zv postgres-svc 5432
```

Monitoring & Auditing Network Flows



Cilium Hubble

- Real-time flow visibility
- Policy verdict logging
- Service dependency maps



Calico Flow Logs

- Allow/deny log entries
- Export to SIEM systems
- Policy recommendation engine



AWS VPC Flow Logs

- ENI-level traffic capture
- CloudWatch + Athena queries
- Pod IP correlation

Recommendation: Enable flow logging *before* enforcing policies. Write policies based on observed traffic, not assumptions.

Network Policy Lifecycle

Recommended Workflow

Treat network policies as code — version, review, test, and automate via CI/CD.

1. **Observe:** Enable flow logging and map actual traffic patterns
2. **Draft:** Write policies based on observed flows, store in Git
3. **Validate:** Lint with `kubeval` or `kubectl --dry-run=server`
4. **Test:** Apply in a staging namespace with connectivity tests
5. **Enforce:** Apply to production with monitoring enabled
6. **Audit:** Periodically review denied flows and policy coverage

Module 8 Summary

Policy Fundamentals

- Kubernetes flat network = default allow
- NetworkPolicy controls L3/L4 traffic
- podSelector targets, policyTypes declares
- Policies are additive (union of allow rules)
- Empty selector = all pods in namespace

Operational Strategy

- Default deny first, allow rules second
- Always permit DNS in egress policies
- Verify CNI enforces NetworkPolicy
- Observe traffic before enforcing
- Treat policies as code in CI/CD



Lab 8: Network Policies

Hands-On Exercise

- Deploy a multi-tier application (frontend, API, database)
- Verify default allow — all pods can reach all pods
- Apply default-deny ingress and egress policies
- Create targeted allow rules for the three-tier communication path
- Apply default-deny egress and allow DNS egress policies
- Debug a broken policy (wrong selector and port)
- Test cross-namespace isolation

Duration: ~30-40 minutes

Key Takeaways

1. Default deny is non-negotiable. Apply default-deny ingress and egress to every namespace.

2. Always allow DNS. Every egress policy must include UDP/TCP port 53. This is the most common cause of policy-related outages.

3. Know your CNI. Standard NetworkPolicy is L3/L4 only. For L7 filtering, DNS-based rules, or global policies, use Calico or Cilium extensions.

4. Observe before enforcing. Enable flow logging and map real traffic before writing policies.

? Questions & Discussion

Module 8: Network Policies

Up Next: Module 9 — Observability — Logging, Metrics & Tracing